

COSC 3P91 – Advanced Object-Oriented Programming

Laboratory 7: Database Integration with JDBC

Department of Computer Science
Brock University

Winter Term

Overview

While Laboratory 5 focused on robustness and managing the unpredictability of runtime environments through exception handling, Laboratory 6 transitions the focus toward data persistence and performance through relational database management systems (RDBMS). This laboratory introduces Java Database Connectivity (JDBC), a standard Java API for accessing tabular data. Students will move beyond flat-file processing to explore the JDBC architecture, which facilitates connecting to a database, sending SQL queries, and retrieving results through specialized drivers. This lab emphasizes the importance of maintaining data integrity and transactional control in production-grade Java applications.

Learning Objectives

By completing this laboratory, students will be able to:

- Integrate relational databases into Java applications using the JDBC API.
- Differentiate between 2-tier and 3-tier JDBC architectures and their respective data flow paths.
- Implement programmatic access to DBMS using specialized JDBC drivers, specifically Type 4 native-protocol all-Java drivers.
- Apply relational integrity rules including unique identifiers, non-repeating column values, and null-value handling.
- Utilize `PreparedStatement`s to improve performance through precompilation and secure SQL execution.
- Manage complex transactions by controlling commit and rollback operations to ensure data atomicity.

Given Problem Description: The "Persistent Coffee Management System"

You are tasked with upgrading the existing data management utility into a system that utilizes a MariaDB or MySQL backend for persistent storage.

- **Schema Design:** Design a relational schema for a `COFFEES` table that enforces integrity rules: distinct rows via unique IDs and strict column constraints.
- **Connectivity:** Implement a connection manager that utilizes a `DriverManager` to load the appropriate JDBC driver and establish a stable connection string.
- **Atomic Transactions:** The system must perform interdependent updates (such as updating sales and total quantities) as a single unit of work. If any part of the operation fails, the system must perform a rollback to maintain data consistency.
- **Efficient Retrieval:** Use `ResultSet`s and cursors to iterate through and display database records efficiently.

Analysis Tasks

Students must answer the following questions to verify their understanding of the implementation:

1. Contrast the 2-tier and 3-tier JDBC architectures. In which scenario is a "middle tier" component required?
2. Why is a Type 4 (Native-Protocol All-Java) driver generally preferred over a Type 1 JDBC-ODBC bridge for modern applications?
3. Explain how `PreparedStatement` objects contribute to both performance and security compared to standard `Statement` objects.
4. Describe the role of `setAutoCommit(false)` in managing a transaction that involves multiple SQL statements.
5. How does a `JdbcRowSet` improve upon the standard `ResultSet` in terms of caching and filtering?

Required Task

The solution must:

1. Design and execute a SQL script to create the required relational schema with appropriate primary keys.
2. Implement a Java class that establishes a connection to a database using a JDBC Driver Manager.
3. Create a "Fail-fast" validation method that ensures data is correct before attempting a database insert.
4. Implement a transaction-based method that updates multiple table rows, using `try-catch` blocks to handle `SQLException` and execute a `rollback()` on failure.
5. Use `PreparedStatements` for all CRUD operations to ensure the system is optimized for repetitive queries.

Key Takeaways

This laboratory emphasizes that robust Java applications must integrate seamlessly with relational database management systems for professional data persistence. By mastering the JDBC architecture and utilizing `PreparedStatements`, developers can create applications that are both performant and secure. Furthermore, understanding transaction management—specifically the "all or nothing" principle—ensures that the application remains reliable even during complex data modifications.